

Chapter 2. Types and Variables

A good name is rather to be chosen than great riches.

—Proverbs 22:1

Computer programs consist of commands (often shorthand as *code*) and data. This chapter introduces the basics of how computers handle data, and how Python is a bit different from many programming languages.

A Computer

What's in a typical computer?

A central processing unit (CPU), or chip

The brains of the operation.

Most computers now have multiple *cores* per CPU, and we'll say more about this in Chapters [23](#) and [27](#).

Random access memory (RAM)

The fast but limited storage area for data and programs.

Cache memory

Like RAM, but smaller and faster.

A disk

Stores much more data than RAM does but is thousands of times slower.

Local input devices

Keyboards, touchscreens, mice, cameras, and microphones.

Local output devices

Displays and printers.

A network

Communicates with other computers, either through cables or the air.

The CPU, RAM, and cache are *volatile*: they work only while electricity is flowing through them. The disk is *nonvolatile*, retaining its data even when the whole computer is turned off. A continual struggle occurs between RAM/cache (fast but limited) and disk (slow but roomier) to balance a computer's performance.

Computer programs and data are stored on the disk and transferred to RAM when the CPU needs to access them. The CPU *fetches* (reads) from RAM and *stores* (writes) back.

In this book, I'll liken RAM to a series of bookshelves, where each rack is the same height and width, and each slot on a particular shelf is uniquely numbered. Each location is independently addressable, so you can put data on a shelf and find it later. Each slot is a *byte* wide. So I'd better explain what a byte is and get back to this bookshelf metaphor afterward.

In [Chapter 27](#), a more complete computer architecture is discussed.

Bits and Bytes

At the very bottom, all data in a computer consists of bits. A *bit* is a tiny unit of storage that can represent one of two states. Way down in the electronics of the computer's storage (RAM or disk), bits are implemented by microscopic components and different voltages.

These two states can be interpreted in different ways, such as these:

- Set or unset
- On or off
- A number (1 or 0)
- A *Boolean* value (true or false)

Figures [2-1](#) and [2-2](#) depict the two bit possibilities.

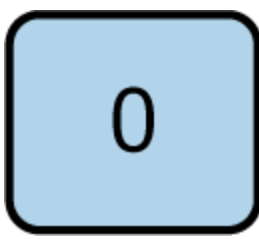


Figure 2-1. Unset bit

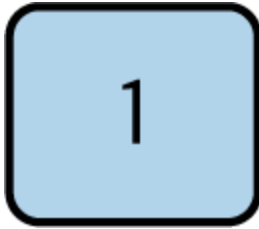


Figure 2-2. Set bit

It's hard to express more complex ideas with just two values, so, going up one level, computers bundle eight bits together into a *byte*. Because each bit has two possible values, and we're now treating eight bits as a unit, there are 2^8 (256) possible combinations of bit values in a byte.

Individual bits or bytes have no actual meaning. Each one has a combination of states, and we somehow need to keep track of them and treat them in a certain way. The bits in a byte have a particular *order*, from *least significant* (bottom) to *most significant* (top). [Figure 2-3](#) shows a byte with the integer value 0, [Figure 2-4](#) represents the integer 1, and [Figure 2-5](#) has all bits set, which could represent the decimal integer 255.

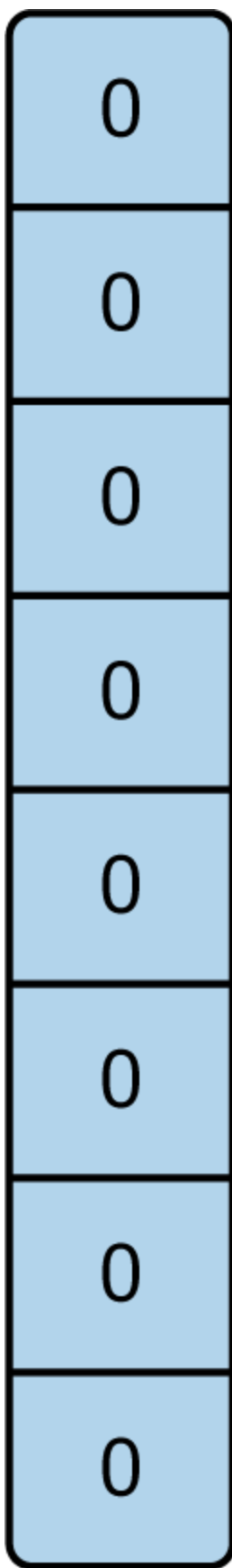


Figure 2-3. Byte with value 0

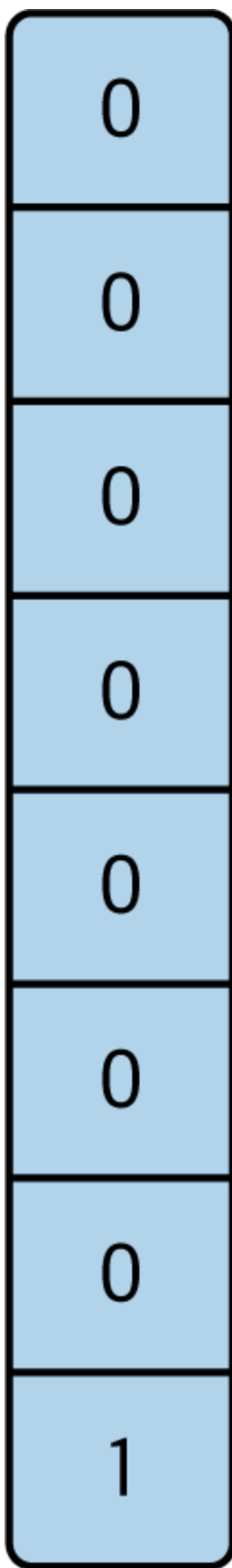


Figure 2-4. Byte with value 1

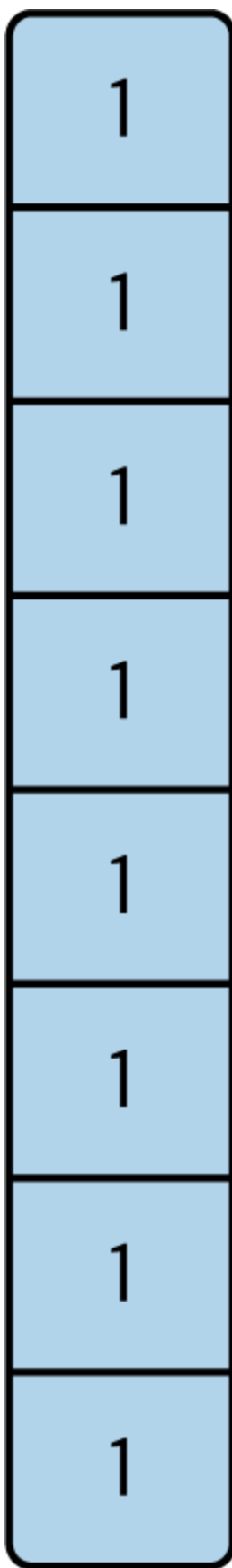


Figure 2-5. Byte with value 255

Because a byte can have up to 256 distinct combinations of bits, we can use base 2, or *binary arithmetic*, to treat a byte as a tiny number. If all its

bits are 0 (off), as in [Figure 2-3](#), that could represent the integer 0. If all bits are 1 (on), as in [Figure 2-5](#), that could represent the decimal integer 255. The least significant bit represents how many 1s, the next bit up represents how many 2s, and the top, most significant bit represents how many 128s. (This is like our familiar base-10 (*decimal*) system, where we have digits from 0 to 9, but no single digit for 10. Instead, we have a 1 in the ten's *place*, and up to the hundred's place, and so on.) These are called *positional systems*. Setting all the bits means (from the top down):

$$128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 255.$$

These examples use *unsigned* (always positive) numbers, but we could treat a byte as a *signed* number (positive, negative, or zero) by using one of the bits to indicate the sign. But then we have one less bit to represent the number's *magnitude*; we can't get to positive 255 in a single byte anymore.

Finally, we could use a byte to represent a text *character*, such as a letter, digit, or punctuation symbol. English uses 26 letters, so 26 lowercase letters, 26 uppercase letters, and the digits 0 through 9 would need only 62 distinct values to fit in a byte. The rest of the 255 possible bit combinations could be used for something else.

The American Standard Code for Information Interchange (ASCII) standard was defined many decades ago to fill in these slots. It also included common punctuation symbols and some nonprinting values. It handled everything you'd see on a typewriter or computer keyboard in a single byte—but only if it was an American keyboard, because that's what the *A* in *ASCII* stands for. In [Chapter 17](#), you'll see how *Unicode* allows all the world's languages (and symbols and even emojis) to be expressed in computers. Unicode has millions of unique symbols; we can't stuff them all into a single byte anymore.

Multibyte Types

Now, let's go back to the bookshelf metaphor that I used earlier. If you have some data that fits inside a byte, you could store it at a specific location on that shelf (location in the computer's memory).

Just as bits can be combined into bytes to represent more possible values, bytes can be combined into adjacent multibyte structures. Two bytes provide $2^8 * 2^8 = 2^{16} = 65,536$ unique bit combinations. Four bytes (32 bits) allows 4,294,967,296 combinations. So we can use more than one adjoining byte slot on our memory bookshelf.

In [Figure 2-6](#), four bytes in a row make a single four-byte number, with the value 1.

0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	1

Figure 2-6. Four bytes, with value 1

The *most significant byte* is the left-most one, so setting only the lowest bit in the least significant (rightmost) byte represents the number 1.

What would a decimal million (1,000,000) look like, represented in bits across four bytes?¹ See [Figure 2-7](#).

In a library, a narrow book might be a novella or poetry collection, and a wider one might be a full novel or nonfiction book like a dictionary. In the computer, we might need only a narrow slot (a byte) for one piece of

data, but a wider slot for something else. Unlike the library, where books have spines with titles and other identifying information, computer memory locations are anonymous. It's the job of the computer program to associate a *type* with memory locations.

0	0	0	0
0	0	1	1
0	0	0	0
0	0	0	0
0	1	0	0
0	1	0	0
0	1	1	0
0	1	0	0

Figure 2-7. Four bytes with the decimal value 1,000,000

Variables

This giant bookshelf of bytes in computer memory contains everything that the computer's brain (its CPU) needs in order to do computery things. But how do you keep track of everything?

A Python program has code (the instructions) and data (the values and their types). Like most computing languages, Python has the concept of a *variable*: a unique name inside the code that refers to a specific data value. It keeps track of exactly where the data is stored on these long memory bookshelves.

Python handles variables differently from many other computer languages, so let's explain, with text and pictures.

Assign a Value to a Variable

You *assign* a value to a variable when you're creating it. In Python, like many other languages, you assign a value by specifying the following:

```
variable = value
```

For example:

```
x = 5
```

assigns the integer value `5` to the variable `x`.

NOTE

This `=` is not expressing *equality*, like a formula in algebra. Maybe computer languages could have used a different symbol for assignment than `=` (and some do), but assignment is so common in computing that it's established itself. You quickly get used to it.

Try assigning the integer value `5` to the variable `x` in the Python shell:

```
$ python
Python 3.13.0 [version info]
Type "help", "copyright", "credits" or "license" for more information.
>>> x = 5
>>> x
5
```

When you type just a variable on a line, the shell prints its value for you on the next line. This is a feature of the shell, and it won't happen when you execute a Python file.

Remember those memory bookshelves? One way to visualize this is that somewhere in memory, one or more bytes are used to represent the integer value `5`. [Figure 2-8](#) uses four contiguous bytes.

0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	1
0	0	0	0
0	0	0	1

Figure 2-8. Four bytes, with value 5

OK, what about the variable `x` ?

That's somewhere else in memory, along with all the code and other variables for this program.

Now, here's an important distinction: in *many* programming languages (but *not* Python), a variable points to the value directly in memory. The variable in those languages can be thought of as a box that can contain values of the same type. The variable is essentially an *address* or a *pointer*, and any valid value can be assigned to its memory destination slot and replace the previous value. Computer languages that work this way are called *statically typed*. They remember where `x` points, and that its val-

ue in memory needs to be a valid four-byte integer.

But, as I said, Python is different. Instead of tracking the type with the variable in the code, the *value* in memory is bundled with the type and extra bookkeeping info. This means that the variable is just a *name*, like a sticky note that you attach to something. Besides a *name*, a Python variable might also be called a *reference* or *label*. Python variables are *dynamically typed*.

[Figure 2-9](#) is a simplified idea of what a Python *object* looks like in memory.

NOTE

Many Python discussions mention that every data type in Python is implemented as an *object*. The problem is that *object* can have multiple meanings. You'll see how to create new data types with classes and objects in [Chapter 11](#). When I use the term here, I mean a minimal data structure that describes the core features of this particular data, including the raw value that the CPU uses.

I'll drop the little bit boxes now and just say what's stored at each location ([Figure 2-9](#)). The actual number of bytes that each component takes up is an implementation detail that we don't need to care about.



Figure 2-9. A Python object in memory

Here's what these parts represent:

- The `id` is unique for each object in memory. Although the Python interpreter may use just the object's location in memory, don't assume this. The important point is that the `id` is unique. The `id()` function returns the unique ID:

```
>>> x = 5
>>> id ( x )
4488951848
```

- The `type` box indicates the Python type of this data location. Besides the built-in types that I list in [Table 2-1](#) later in this chapter, you can define your own custom types in Python. You'll see how in [Chapter 11](#).
- The actual value bits (in this example, those for the integer `5`) might be stored here in the `value` location or elsewhere in memory. In coming chapters, you'll see data structures that are bigger than single integers and need room to roam.
- What about that `reference count` ? It's how Python tracks the number of variables pointing to the same object. If this count

reaches zero, no variables are referring to this particular piece of data, and the program has no way of reaching it anymore. Python can then clear this area of memory and use it for other values; this is called *garbage collection*.

OK. What all this means is that, in Python, if you type `x = 5`, Python will create that multipart data structure (*object*) in memory with type `integer` and value `5`. Python will return this structure's memory address and associate it with the variable `x`. Then the reference count will be incremented from `0` to `1`, because its new friend, variable `x`, now knows about it.

After this in the program, any reference to the variable `x` will follow the big metaphorical finger that it points to memory and get that lovely integer `5`.

If you like visual explanations, the [YouTube clip “How Variables Works in Python | Explained with Animations” by Sreekanth](#) about variables in Python and other languages may help.

Change the Value of a Variable

The value itself can be changed by assigning a new value to the same variable. Say we make this change:

```
x = 6
```

As I mentioned earlier, in most languages (*not* Python), a variable is a pointer; that old bit pattern representing the value 5 gets wiped, and [Figure 2-10](#) shows that it's overwritten by the bits that represent 6.

Python does this differently: it first creates a new object in memory, with type integer and the value 6. Then it associates that existing variable `x` with this new object, and accordingly increments the reference count of this new object to one.

Because `x` now refers to a new object, its `id` changes:

```
>>> x = 5
>>> id ( x )
4488951848
>>> x = 6
>>> id ( x )
4488951880
```

0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	1
0	0	0	1
0	0	0	0

Figure 2-10. Four bytes, now with value 6

Because the variable `x` no longer refers to the previous value (5), the reference count of the old object (the one with integer value 5) is decremented to zero.

An object with a zero reference count is no longer addressable by the code, so Python eventually *garbage collects* it (frees the memory it uses).

NOTE

A *very* technical exception to this arises for a value like 5. The Python interpreter has a few performance tweaks: it precreates objects for the integers -5 to 255 and keeps them around, because small numbers are so common in code. Any other objects do go on the garbage truck. Au revoir!

Delete a Variable

Another way to get rid of a value is to delete its variable: `del x`.

This also decrements its value object's reference count and causes its garbage collection. You normally don't need to do this in Python; the interpreter keeps track of the *scope* (code extent lifetime) of each variable, zeroing reference counts and freeing used.



Memory management is difficult and notoriously error-prone in many other languages. One of the big advantages of Python is that you don't need to manually manage memory (say this fast three times). There are no C-style `malloc()` or `free()` calls.

I'm presenting all this detail for a reason: to pre-explain some Python behavior that might be confusing if you're used to other languages, such

as assigning a *mutable* (changeable) piece of data to multiple variables. You'll see this in [Chapter 8](#) when we talk about lists.

Name Variables

Python variable names have rules:

- They can contain these characters:²
 - Lowercase letters (a through z)
 - Uppercase letters (A through Z)
 - Digits (0 through 9)
 - Underscore (_)
- They are *case-sensitive*: `thing`, `Thing`, and `THING` are different names.
- They must begin with a letter or an underscore, not a digit.
- They cannot be one of Python's *reserved words* (also known as *key-words*).

The reserved words are listed here:

<code>False</code>	<code>await</code>	<code>else</code>	<code>import</code>	<code>pass</code>
<code>None</code>	<code>break</code>	<code>except</code>	<code>in</code>	<code>raise</code>
<code>True</code>	<code>class</code>	<code>finally</code>	<code>is</code>	<code>return</code>
<code>and</code>	<code>continue</code>	<code>for</code>	<code>lambda</code>	<code>try</code>
<code>as</code>	<code>def</code>	<code>from</code>	<code>nonlocal</code>	<code>while</code>
<code>assert</code>	<code>del</code>	<code>global</code>	<code>not</code>	<code>with</code>
<code>async</code>	<code>elif</code>	<code>if</code>	<code>or</code>	<code>yield</code>

Within Python, you can find the reserved words with

```
>>> help ( "keywords" )
```

or

```
>>> import keyword
>>> keyword . kwlist
```

These are valid names:

- a
- a1
- a_b_c___95
- _abc
- _1a

These names, however, are not valid:

- 1
- 1a
- 1_
- name!
- another-name

Follow Naming Conventions

Python also has *conventions* when naming variables. They aren't rules, like those I just mentioned, but they'll help you be consistent with other Python code out there:

- Python variables should use *snake case*: lowercase letters (and possibly digits) separated by the underscore (`_`) character. Examples:
`x_squared , num_ghosts .`
- Other languages (like Java and JavaScript) prefer *camel case* (an initial lowercase letter and uppercase humps inside): `xSquared , numGhosts .`
- Other languages capitalize the first letter too: `XSquared , NumGhosts .` Python also likes this convention when you're defining *object classes*, which are coming in [Chapter 11](#). Sometimes this is called *Pascal case* or *upper camel case*.
- Although Python doesn't have true *constants* (variables whose value can't change), it recommends snake case with all caps to help everyone remember that this variable should *not* be modified after its initial assignment:
`MAX_ITEMS , SECRET_CODE .`

Python really seems to like that humble underscore character:

- A name that starts with a single underscore (`_`) is treated as sort of private by the `import` statement (see [Chapter 12](#)).
- A name that starts with two underscores (`__`) is treated specially

when creating object classes (see [Chapter 11](#)).

- Names that start *and* end with double underscores are used for so-called *magic*, or *dunder*, methods in object classes (also in [Chapter 11](#)).

Python Types

You’ve seen that bits, bytes, and multibyte combinations have no inherent meaning in computer memory or storage. Something, somewhere, assigns that meaning and remembers it. That’s where computer language *types* come in.

Each computer architecture (a specific design from a computer company) has the ability to handle particular bit combinations and treat them as distinct types. These types include numbers of various sizes, text characters, and so on. Python defines its own object types to match these common hardware types, as well as some software-only types like complex numbers. [Table 2-1](#) lists these built-in types, with table columns that contain the following:

- The Name column contains equivalent English names.
- The Type column indicates the actual Python wording for this type.

- The Mutable column indicates whether the *value* of this type (not the type itself!) can be changed.
- The Examples column shows some Python syntax to express values of this type.
- The Chapter column is where you'll really get into the details.

Table 2-1. Python's basic data types

Name	Type	Mutable?	Examples	Chapter
Boolean	<code>bool</code>	No	<code>True</code> , <code>False</code>	Chapter 3
Integer	<code>int</code>	No	<code>47</code> , <code>2500</code> , <code>0</code> , <code>25_000</code>	Chapter 3
Floating point	<code>float</code>	No	<code>3.14</code> , <code>2.7e5</code>	Chapter 3
Complex	<code>complex</code>	No	<code>3j</code> , <code>5 + 9j</code>	Chapter 3
Text string	<code>str</code>	No	<code>'alas'</code> , <code>"alack"</code> , <code>'''a verse attack'''</code>	Chapter 4
List	<code>list</code>	Yes	<code>['Winke', 'Blinken', 'Nod']</code>	Chapter 8
Tuple	<code>tuple</code>	No	<code>(2, 4, 8)</code>	Chapter 8
Bytes	<code>bytes</code>	No	<code>b'ab\xff'</code>	Chapter 5
Bytearray	<code>bytearray</code>	Yes	<code>bytearray(...)</code>	Chapter 5
Set	<code>set</code>	Yes	<code>set([3, 5, 7])</code>	Chapter 9
Frozen set	<code>frozenset</code>	No	<code>frozenset(['Elsa', 'Otto'])</code>	Chapter 9

Name	Type	Mutable?	Examples	Chapter
Dictionary	dict	Yes	<code>{ 'game': 'bingo', 'dog': 'dingo', 'drumme r': 'Rin go' }</code>	Chapter 9

In the next chapter, we'll get into number types: bool, int, float, and complex.

Specify Values

By now, you've seen that a value can be specified as a *literal* (a direct value, like `5`) or a *variable* (like `x`, which was assigned the value `5`). Python has rules for specifying literal values, and they depend on the underlying types. As you've seen, an integer is specified as a sequence of digits, but a *float* (floating-point value, which you'll see in the next chapter) can consist of a string of digits and a decimal point.

In the next few chapters, I'll show how to specify literal values for Python's standard types, assign them to variables, and perform various *operations* (like the addition of numbers).

Objects as Plastic Boxes in Memory

You can think of an *object* as a clear plastic box on that memory shelf. Some content has a fixed size (ID, type, reference count), and other content varies in size (the value bits themselves). All Python *data* is like this. You could have an `int` object sitting next to a `list` object, and a user-defined object next to that.

Python *code* lives somewhere else in memory (on another shelf). A *variable* in Python code can be pictured as a sticky note attached to its data object, or a tag on a thread taped to it. Python keeps track of all this code and data so we don't have to.

Review/Preview

This chapter focused on variables (names used in programs) and objects. Python is different from many programming languages, because the variable is just a name, and the object it refers to includes other information: a unique ID, type, value, and reference count of the number of variables that refer to it.

Coming next: numbers! Yay! The simplest objects of all.

Practice

2.1 Install Python if you don't have it on your computer already. Use the most recent version if you can.

2.2 Start the interactive interpreter and type **print(42)** . The interpreter should echo 42 on the next line.

2.3 If you're still in the interactive interpreter, type **43** . The interpreter should print 43 on the next line. This is a feature of the interactive interpreter only and won't print anything if you're executing a Python file.

2.4 Try assigning the literal value 7 to variables with different names. Try some illegal names to see the error messages that Python tosses back to you.

1 A quick peek at two Python ways to convert that decimal 1,000,000 to binary for this image: `bin(1000000)` or `f"{1000000:b}"` .

2 Technically, they can also be some Unicode characters, but this can get [complicated](#).